

Programming Lab 04: Exception Handling

V. Markos
Mediterranean College

Fall, 2024 – 2025, Week 08

Abstract

What is this? Probably an ongoing lab for this course, where you will create simple game in Python. Do not expect spectacular graphics and all that stuff, since this course is not about that, but expect to encounter and attack several data structures and algorithm-related problems as we go into more sophisticate designs for our game. Stay tuned for more...

1 Last Time Recap

Last time we extended our game with the following features (at least):

- Exporting our game session into a text file, including all relevant information that should be kept.
- Importing our game session from a text file, utilising the text file we generate on export.
- Proper adjustments regarding informing the user about relevant functionalities, by implementing a help page for our game.
- Maybe also allowed for importing from / exporting to JSON.

2 Invalid User Input

As you might well have experienced so far, we have handled invalid user input in a simplistic way, by repeatedly asking the user to provide a valid command to execute. However, it is more consistent and meaningful to check for valid or invalid user input by making use of exceptions. So, here comes your first task;

Task 1: Handling Invalid User Input

Handle user input by properly utilising Python exceptions. That is, your (or any) function that handles user input should throw an exception in the case of invalid commands and then properly handle those exceptions outside that function. You can either choose to implement your own exceptions or utilise Python's already implemented functions appropriately. Both ways can work equally well, but keep in mind that, as a rule of thumb, when there exists a native exception that suits your purpose, you should better use this instead of your own custom one.

3 More Exceptions

In our previous version of the game, in order to handle non-move user input, we had to check whether the output of a certain function was:

- either `None`, which we decided would mean that an invalid move had been provided;
- or a valid move string, which we would then translate into a valid move sequence.

One might complain that this makes our function a bit inconsistent in the sense of it returning values of different type (strings and `NoneType`). So, your next task is mostly related to re-factoring your code so as to make user input reading more clear

Task 2: Exceptions And Function Output

Refactor your code in a more meaningful way when it comes to handling user input. Consider that, so far, we have implemented (at least) the following types of user input:

- Moving the main character;
- Performing actions besides gameplay, like exporting or importing, quitting and asking for help;
- Providing invalid input.

So, you should handle all of them in a more structured way, e.g., by using different functions for each purpose. Also, for each case separately, you have to properly handle any exceptions that might occur, e.g., in case of invalid or malformed user input. For instance, a user writing `“up”` might be handled differently (since this clearly maps to moving “up”) than a user typing nonsense.

4 Further Considerations

So far, we have handled exceptions in almost all obvious cases. However, there might be places where you should check for errors that we haven't caught in the above. So, here comes your first optional task:

Task 3: Error Handling Reloaded

As with your previous tasks, look for places where runtime errors might occur, in which case you should intervene and handle them properly. That is, you should probably utilise Python's exceptions to make sure that all errors do not interrupt program execution but are probably handled within the program.

Having handled any potential errors gracefully, you have already accomplished a big portion of making your game safer to run. An additional step towards that direction should be to keep track of any errors — and, even, user input — and store them in what we usually call **log files**. So, here comes your next optional task:

Task 4: Logging

As you are capturing errors and user input, implement any related functionality that logs them into log files. You can choose among any option you might see more fitting for your case. Below follow some choices you might find useful or not:

- Use a single text file to keep track of all user input at the time it happens.
- Use separate files for errors and valid user input.
- Store metadata such as the timestamp of each command and / or error, as well as anything else you might find relevant.
- Use Python's libraries, such as **logging** to facilitate the logging process and handle much stuff automatically.